

Internship Project Report



EXPERIMENTAL STUDY OF ADC AND DAC INTERFACING **WITH RASPBERRY PI 5 USING SPI**

Sachindra P

PSG College of Arts & Science, Coimbatore

Under the guidance of:

Ghanshyam Kumar

[GUIDE DESIGNATION]



Space Astronomy Group

URSC, ISRO

02 Jul 2026 – 14 Aug 2026

CERTIFICATE

This is to certify that the project report titled “**Experimental Study of ADC and DAC Interfacing with Raspberry Pi 5 Using SPI**” is a Bonafide work carried out by **Sachindra. P**, M.Sc. Applied Electronics (24MEL008), Department of Electronics, PSG College of Arts & Science, Coimbatore, under my guidance and supervision at the **Space Astronomy Group, U R Rao Satellite Centre (URSC), Indian Space Research Organisation (ISRO)**, during the period of the internship program.

The work presented in this report is original and has been carried out with dedication and sincerity. The project demonstrates a practical understanding of ADC and DAC interfacing, SPI communication, digital data transfer, and signal verification using Raspberry Pi 5.

To the best of my knowledge, this work has not been submitted elsewhere for any other degree or diploma.

Ghanshyam Kumar

Mentor

Space Astronomy Group

U R Rao Satellite Centre (URSC)

Indian Space Research Organisation (ISRO)

Date: 14.8.2026

Place: Bangalore

OFFICIAL CERTIFICATE / SIGNATURE DETAILS

DECLARATION

I, SACHINDRA.P, M.Sc. Applied Electronics student of PSG College of Arts & Science, Coimbatore, hereby declare that the internship project report titled **“Experimental Study of ADC and DAC Interfacing with Raspberry Pi 5 Using SPI”** is based on the work carried out by me at [ISRO / ISITE / SAG OFFICIAL WORDING] under the guidance of Ghanshyam Kumar.

I declare that the report represents my internship learning and the work actually performed during the stated internship period. Experimental readings, circuit details, measurements, observations, and technical specifications are to be included only when supported by the laboratory work, supplied documents, or verified technical references.

1. The sources and assistance used during the preparation of the report will be appropriately acknowledged.
2. The report does not intentionally contain fabricated experimental results or unsupported claims.
3. The report has not been submitted previously for another academic qualification unless otherwise stated by the institution.
4. The report will be submitted in accordance with the applicable college and internship guidelines.

Sachindra P

Student

PSG College of Arts and Science

Coimbatore, Tamil Nadu

Date: 14.8.2026

Place: Bangalore

STUDENT SIGNATURE

Sachindra. P

TABLE OF CONTENTS

Certificate	2
Declaration	3
1. Introduction	5
2. Project Aim and Objectives	6
3. Hardware Description	7
4. SPI Protocol	8
5. Pin Connection Table.....	9
6. ADC128S102	9–10
7. DAC121S101	10
8. Software Environment	11
9. Program Implementation	11
10. Experimental Procedure	12
11. Calculations and Data Transfer Analysis	12
12. Data Transfer Analysis	13
13. Results and Discussion	13
14. Learning Outcomes	14
15. Applications	14
16. Future Scope	14
17. Conclusion	15

1. INTRODUCTION

Modern electronic systems frequently require conversion between physical analog quantities and digital information. Sensors, measurement instruments, control systems, and embedded platforms use analog-to-digital converters (ADCs) to acquire signals and digital-to-analog converters (DACs) to reconstruct or generate analog signals.

An ADC converts an analog input into a numerical digital representation. The digital representation can then be processed by a computing platform. A DAC performs the complementary operation by converting a digital code into an analog output. The practical behaviors of these devices depends not only on their conversion characteristics but also on the communication interface, reference conditions, power arrangement, grounding, and measurement method.

The present internship work used a Raspberry Pi 5 as the computing platform. Communication with the ADC128S102 and DAC121S101 was performed through the Serial Peripheral Interface (SPI). Python and the spidev library were used to configure the SPI interfaces, transfer data, process the received ADC code, and send digital data to the DAC.

1.1 Objectives

- Understand ADC interfacing with Raspberry Pi 5.
- Understand DAC interfacing with Raspberry Pi 5.
- Study SPI protocol and practical data transfer.
- Acquire ADC data and reconstruct the digital code.
- Convert the ADC code into an estimated voltage in software.
- Transfer digital data to the DAC.
- Observe and verify the resulting analog signal using measurement equipment.

1.2 Scope

The scope of the internship work is limited to interfacing, communication, testing, observation, and understanding of the ADC/DAC system. It does not claim development of a new ADC, DAC, or hardware design unless additional work is subsequently documented.

2. PROJECT AIM AND OBJECTIVES

Aim: To interface ADC128S102 and DAC121S101 devices with a Raspberry Pi 5 and understand SPI-based data acquisition, digital data transfer, and digital-to-analog conversion.

3.1 Objectives

- Configure the ADC and DAC SPI interfaces.
- Acquire ADC data from the selected input channel.
- Understand the ADC conversion sequence used by the implementation.
- Reconstruct the digital code from received bytes.
- Calculate the corresponding input voltage in software.
- Transmit the digital code to the DAC.
- Calculate the expected DAC output from the digital code.
- Verify the system through external voltage application and measurement.

4. SYSTEM OVERVIEW

The experimental signal flow can be represented as follows:

**External Variable Voltage → ADC128S102 → SPI → Raspberry Pi 5 → SPI → DAC121S101
→ Analog Output**

The external variable supply provides the analog test input. The ADC samples and converts this input into digital data. The Raspberry Pi receives the digital data through SPI and processes it using Python. The same digital value is then transferred to the DAC, which produces an analog output according to its reference and conversion characteristics.

3. HARDWARE DESCRIPTION

5.1 Raspberry Pi 5

The Raspberry Pi 5 serves as the processing and communication platform. Its SPI interface provides the serial communication path to the converter devices. The exact operating-system version and other configuration details should be entered from the actual laboratory setup.

5.2 ADC128S102

ADC128S102 is the ADC used for analog input acquisition. Channels IN0, IN1, and IN2 were identified for testing. The external variable supply was used as the input source, with approximately 0–1 V tested. Datasheet-level resolution, timing, reference requirements, and command format should be inserted from the verified device documentation.

5.3 DAC121S101

DAC121S101 is the DAC used to convert the transferred digital code into an analog output. The implemented software calculation uses a 3.3 V reference. Datasheet-level resolution, input format, timing, and reference requirements should be verified before final submission.

4. SPI PROTOCOL

Serial Peripheral Interface (SPI) is a synchronous serial communication protocol commonly used for communication between a controller and peripheral devices. The controller supplies the clock and selects the required peripheral using a chip-select signal.

- SCLK – serial clock generated by the controller.
- MOSI – serial data transmitted from controller to peripheral.
- MISO – serial data returned from peripheral to controller.
- CE/CS – chip-select signal used to select the intended peripheral.
- SPI supports simultaneous transmission and reception, depending on the device implementation.

6.1 SPI Modes

SPI timing is determined by clock polarity (CPOL) and clock phase (CPHA), producing four standard modes. The actual modes in the supplied implementation are ADC mode 3 and DAC mode 1. These settings must be compared with the official device datasheets before final submission.

6.2 Raspberry Pi Device Selection

The supplied program opens the ADC using `adc.open(0, 0)` and the DAC using `dac.open(0, 1)`, corresponding to separate chip-select interfaces in the `spidev` configuration.

6.3 Project Data Transfer

The Raspberry Pi initiates the SPI transaction, sends the required bytes, receives ADC data where applicable, reconstructs the digital value, and subsequently transmits the value to the DAC.

PIN CONNECTION TABLE

Table 11.1: SPI and Power Connections Used for the Experimental Setup

Raspberry Pi 5 Function	GPIO	Physical Pin	ADC128S102	DAC121S101	Connection / Function
SPI0 MOSI	GPIO10	19	Pin 14 – DIN	Pin 4 – DIN*	Digital data from Raspberry Pi
SPI0 MISO	GPIO9	21	Pin 15 – DOUT	—	Digital data from ADC to Raspberry Pi
SPI0 SCLK	GPIO11	23	Pin 16 – SCLK	Pin 5 – SCLK*	SPI clock
SPI0 CE0	GPIO8	24	Pin 1 – CS	—	ADC chip select
SPI0 CE1	GPIO7	26	—	Pin 6 – SYNC*	DAC chip select
3.3 V Supply	—	17	Pin 13 – VD	—	ADC digital supply
5 V Supply	—	2 / 4	Pin 2 – VA	—	ADC analog supply/reference**
Ground	—	6 / 9 / 14 / 20 / 25 / 30 / 34 / 39	Pin 3 – AGND; Pin 12 – DGND	Pin 2 – GND*	Common ground
Analog Input	—	External	Pin 4 – IN0	—	External variable voltage input
Analog Input	—	External	Pin 5 – IN1	—	External variable voltage input
Analog Input	—	External	Pin 6 – IN2	—	External variable voltage input
Analog Output	—	—	—	Pin 1 – VOUT*	DAC analog output

6. ADC128S102

The ADC section of the project demonstrates the conversion of an externally supplied analog voltage into a digital code. The program accepts a channel number and constructs a two-byte transmission sequence.

7.1 Channel Selection

The supplied program uses: $tx = [(channel \& 0x07) \ll 3, 0x00]$. The channel number is first limited using a bit mask and then shifted before being placed into the transmitted byte. The exact interpretation of these bits must be read together with the ADC datasheet.

7.2 Conversion Sequence

The implementation performs a first SPI transfer whose result is discarded and then performs a second transfer whose received bytes are used for the digital-code reconstruction. This sequence should be described as the experimental software implementation and compared against the official conversion sequence.

7.3 Digital Code Reconstruction

The data received is reconstructed using:

$$\text{value} = ((\text{rx}[0] \& 0x0F) \ll 8) | \text{rx}[1]$$

The first received byte is masked to retain the lower four bits. Those bits are shifted left by eight positions and combined with the second byte using the bitwise OR operation. This reconstructs a multi-bit numerical value from the received bytes.

7.4 ADC Voltage Calculation

$$\text{ADC Voltage} = \text{ADC Digital Value} \times 5.0 / 4096.0$$

The equation above is the calculation implemented in the supplied Python program. The reference-voltage assumption and denominator should be verified against the actual device configuration before final submission.

7. DAC121S101

The DAC section receives the digital code produced from the ADC stage and transfers it through SPI. The code is first limited to the intended 12-bit range using a mask.

$$\text{value} \&= 0x0FFF$$

The 0x0FFF mask retains the lower 12 bits of the value. The resulting value is separated into a high byte and low byte before SPI transmission.

$$\text{high_byte} = (\text{value} \gg 8) \& 0xFF$$

$$\text{low_byte} = \text{value} \& 0xFF$$

The supplied program sends a preliminary two-byte transfer and then transmits the high and low bytes. The exact device-specific frame should be verified using the official DAC documentation.

8. SOFTWARE ENVIRONMENT

Python was used as the programming environment. The spidev library provides access to Linux SPI devices from Python. The time module is used to introduce a five-second delay between successive iterations.

The ADC SPI interface is opened on bus 0, chip select 0, with a speed of 1 MHz and mode 3. The DAC interface is opened on bus 0, chip select 1, with a speed of 1 MHz and mode 1. These are the settings in the supplied implementation

9. PROGRAM IMPLEMENTATION

The program begins by importing spidev and time. Two independent SPI objects are created: one for the ADC and one for the DAC.

1. ADC initialization: `adc = spidev.SpiDev(); adc.open(0, 0).`
2. DAC initialization: `dac = spidev.SpiDev(); dac.open(0, 1).`
3. Both interfaces use a configured SPI speed of 1 MHz.
4. The ADC uses mode 3 and the DAC uses mode 1 in the supplied implementation.
5. `read_adc(channel)` constructs the transmission bytes and performs the two SPI transfers.
6. The received bytes are combined into the ADC digital value.
7. `write_dac(value)` limits the value to 12 bits, separates it into two bytes, and transfers the bytes.
8. The main loop repeatedly reads the ADC, calculates an input voltage, writes the same digital value to the DAC, calculates the corresponding DAC voltage, prints the values, and waits five seconds.
9. The finally block closes both SPI devices.

10. EXPERIMENTAL PROCEDURE

1. Prepare the Raspberry Pi 5 and enable the required SPI interface.
2. Connect the ADC128S102, DAC121S101, Raspberry Pi, power supplies, and common grounds according to the actual laboratory setup.
3. Apply an externally supplied variable voltage to the selected ADC input.
4. Run the Python program and initiate SPI communication.
5. Read the ADC digital code from the selected channel.
6. Calculate the input voltage using the implemented equation.
7. Transfer the ADC digital value to the DAC.
8. Calculate the expected DAC output using the implemented DAC equation.
9. Measure or observe the DAC output using the available equipment.
10. Repeat the test for additional channels where applicable.
11. Record the actual readings and compare calculated and measured values.

11. CALCULATIONS AND DATA TRANSFER ANALYSIS

14.1 ADC Calculation

$$V_{ADC} = \text{Code} \times V_{REF} / 4096$$

For the supplied implementation, V_{REF} is represented as 5.0 V. Actual calculations should be completed using the verified reference condition and measured data.

14.2 DAC Calculation

$$V_{DAC} = \text{Code} \times V_{REF} / 4096$$

For the supplied implementation, V_{REF} is represented as 3.3 V. The final report should distinguish this software calculation from the actual electrical reference configuration.

14.3 Error Analysis

$$\text{Absolute Error} = |\text{Measured Value} - \text{Expected Value}|$$

$$\text{Percentage Error} = |\text{Measured} - \text{Expected}| / |\text{Expected}| \times 100$$

12. DATA TRANSFER ANALYSIS

The complete digital path can be described as: analog input → ADC sampling and conversion → ADC digital code → SPI transfer to Raspberry Pi → Python processing → digital code transfer to DAC → DAC conversion → analog output.

Bit masking and shifting are important in reconstructing multi-byte digital values. In the ADC code, the mask 0x0F isolates the lower four bits of the first received byte. The left shift by eight positions moves these bits into their numerical positions, and the bitwise OR combines them with the second byte.

In the DAC code, 0x0FFF limits the transmitted numerical value to 12 bits before the value is separated into high and low bytes.

13.1 Implementation Note

The experimental software configuration should be documented exactly as implemented. If any difference is found between the code and official datasheet requirements, the difference should be reported explicitly rather than silently changing the original code.

14. RESULTS AND DISCUSSION

The results section must be based on actual measurements. The following points should be discussed after the observation tables are completed:

- Whether ADC acquisition operated as expected.
- Whether SPI communication produced the expected data transfer.
- Whether DAC output was generated.
- Relationship between applied voltage and ADC digital code.
- Relationship between transferred digital code and DAC output.
- Agreement between calculated and measured values.
- Behaviour of the tested ADC channels.
- Stability, noise, and observed deviations.

15. LEARNING OUTCOMES

- Practical SPI interfacing and communication.
- Raspberry Pi-based electronics interfacing.
- ADC and DAC concepts and data conversion.
- Python programming for hardware communication.
- Bit masking, bit shifting, and byte reconstruction.
- Laboratory measurement and verification.
- Debugging and interpretation of technical documentation/datasheets.

16. APPLICATIONS

ADC, DAC, and SPI-based systems have broad applications in data acquisition, instrumentation, embedded systems, control systems, sensor interfaces, laboratory test systems, and electronic communication systems. The present project provides practical understanding of these interfaces without claiming deployment of this exact experimental arrangement in a particular ISRO flight system.

17. FUTURE SCOPE

- Automated multi-channel acquisition.
- Graphical user interface and real-time plotting.
- Automated data logging.
- Automated calibration and error analysis.
- Extended voltage-range and long-duration testing.
- SPI signal capture using an oscilloscope or logic analyzer.
- Further software-based monitoring and reporting.

18. CONCLUSION

The internship provided practical exposure to ADC and DAC interfacing, SPI communication, and Raspberry Pi 5-based electronic data acquisition. The project involved interfacing the ADC128S102 and DAC121S101 with the Raspberry Pi 5 and understanding the complete flow of data from an external analog input to digital processing and back to an analog output.

The ADC section helped in understanding the process of acquiring an analog signal and representing it as a digital value. The received SPI data was processed using Python, where bit masking, bit shifting, and byte-wise data reconstruction were used to obtain the ADC digital value. The calculated ADC value was then transferred to the DAC through the SPI interface.

The project also provided practical understanding of SPI signals such as MOSI, MISO, SCLK, and chip-select signals, along with the use of separate chip-select lines for the ADC and DAC. Python and the spidev library were used for SPI configuration, data transfer, processing, and device control.

Overall, the internship strengthened practical knowledge of embedded electronics, digital communication, ADC/DAC conversion, Python-based hardware interfacing, and laboratory testing. The experience also improved understanding of troubleshooting, datasheet interpretation, and systematic verification of electronic systems.

The project therefore provided a valuable practical foundation for further work in embedded systems, data acquisition, instrumentation, and electronic system testing.